If the value in row 0 and column 1 is high, we know that node 0 attends strongly to node 1. The normalized attention (i.e., adjacency) weight matrix $A = softmax(QK^T): \mathbb{R}^{n \times n}$ tells us all the pairwise interactions between nodes. We then multiply this by the value matrix, which will update each node's feature vector according to its interactions with other nodes, such that the final result is an updated node matrix, $\hat{N}: \mathbb{R}^{n \times f}$. We can then pass this updated node matrix through a linear layer to do additional learning over the node features and apply a nonlinearity to model more complex features. We call this whole procedure a *relational module* or *relational block*. We can stack these relational modules sequentially to learn higher order and more complex relations.

In most cases, the final output of our neural network model needs to be a small vector, such as for Q values in DQN. After we've processed the input through 1 or more relational modules, we can reduce the matrix down to a vector by either doing a MaxPool operation or an AvgPool operation. For a node matrix $\hat{N}: \mathbb{R}^{n \times f}$, either of these pooling operations applied over the *n* dimension would result in a *f*-dimensional vector. MaxPool just takes the maximum value along the *n* dimension. We can then run this pooled vector through one or more linear layers before returning the final result as our Q values.

10.3. Implementing self-attention for MNIST

Before we delve into the difficulties of reinforcement learning, let's try building a simple self-attention network to classify MNIST digits. The famous MNIST data set is 60,000 hand-drawn images of digits, where each image is 28 × 28 pixels in grayscale. The images are labeled according to the digit that is depicted. The goal is to train a machine learning model to accurately classify the digits.

This data set is very easy to learn, even with a simple one-layer neural network (a linear model). A multilayer CNN can achieve in the 99% accuracy range. While easy, it is a great data set to use as a "sanity check," just to make sure your algorithm can learn anything at all.